uPyCore: System and Subsystem Diagrams

A microcontroller-class CPU whose instruction set is MicroPython bytecode

uPyCore systems notes
`docs/paper/systems/upycore_architecture.tex`

October 18, 2026

Abstract

uPyCore executes MicroPython v1.29.0 bytecode directly in hardware. Programs are MicroPython source; the host `mpy-cross` compiler produces `.mpy`, a linker packs firmware and program into one flash image, and a Verilator model of `mpy_soc` runs it. There is no on-device compiler. This note collects the figures for the paper: the end-to-end flow, the SoC, the core and its firmware-hook split, the memory system and value encoding, the arithmetic datapath, the garbage collector, and the interrupt/peripheral block. Figures mark blocks that exist in the tree but are not yet wired into `mpy_soc`. Colour key: RTL firmware / immediate host tooling / pointer storage not yet wired / failure.

Contents

1 System overview **1**

2 SoC structure **2**

3 The execution core **2**
3.1 Call and return 2

4 Memory system and value encoding **5**
4.1 Tagged values 5
4.2 Heap object layouts 5

5 Arithmetic datapath **5**

6 Garbage collector **5**

7 Interrupts and peripherals **8**

8 Verification map **9**

1 System overview

Figure 1 shows the path from source to result. The unix oracle (MicroPython built as the `upycore` variant) produces the golden output the simulated hardware must match byte for byte; a second, independent compiler (`vendor/upy2`, `mpyc`) is run through the same suite and byte-compared with `mpy-cross`.

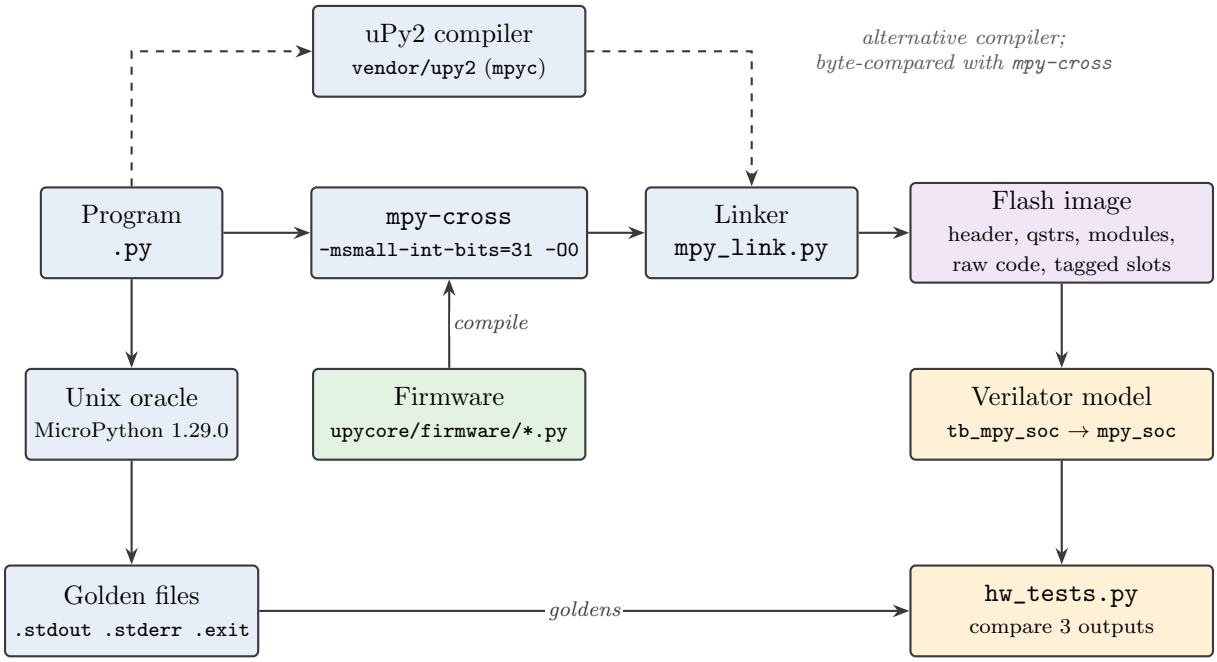


Figure 1: Build and verification flow. Solid arrows are the default `make test` path; dashed arrows are the uPy2 compiler pass (`make test-compiler`).

2 SoC structure

`mpy_soc` instantiates exactly three blocks—`mpy_core`, `mpy_gc`, and `mpy_mem`—on a single memory port. The collector owns the port while `g_busy` is high; the core only waits during an allocation or collection, so the two never contend. Accesses to the simulator-control page are answered inside the SoC and never reach memory.

Table 1: Top-level observability exported by `mpy_soc`.

Port group	Meaning
`con_valid_o con_err_o con_byte_o`	one console byte per pulse, stdout or stderr
`halted_o halt_code_o halt_value_o`	core halt or `SIM_HALT` store (`ABI_HALT_SYSEXIT`)
`cycles_o ops_o`	cycle and retired-opcode counters
`gc_collections_o gc_free_o`	collector statistics
`trace_*`	per-opcode trace: PC, opcode byte, stack pointer

3 The execution core

The core keeps *one frame live in registers*. It executes the common shape of every opcode itself and delegates the rest to *firmware hooks*: MicroPython functions in flash that the core calls like any Python function. Figure 3 gives the structure; Figure 4 gives the dispatch decision.

3.1 Call and return

A call spills the caller's whole frame to a 512-byte record in SRAM and builds the callee's record above it; a return or unwinding exception refills the caller. Thirty-two records fit the reserve; a 33rd faults and software reports `RuntimeError: maximum recursion depth exceeded`.

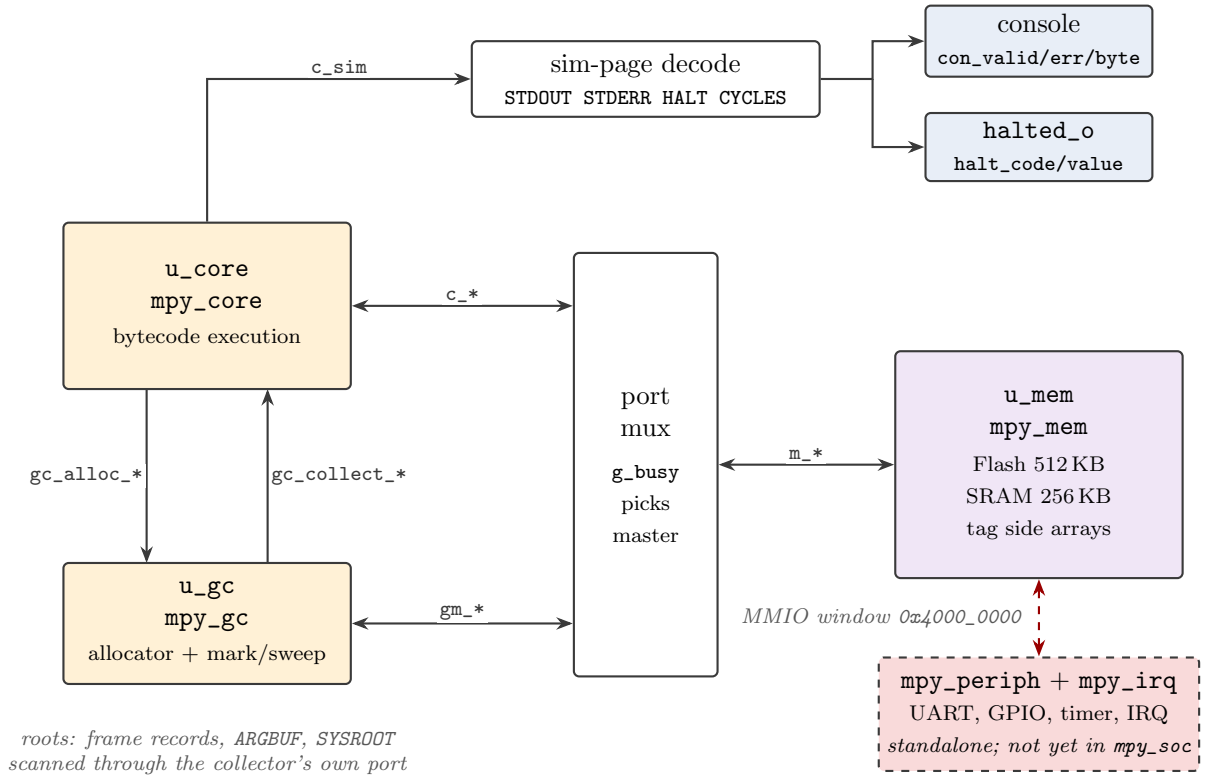


Figure 2: `mpy_soc`. Solid blocks are instantiated; the dashed red block has RTL and a testbench but is not yet connected.

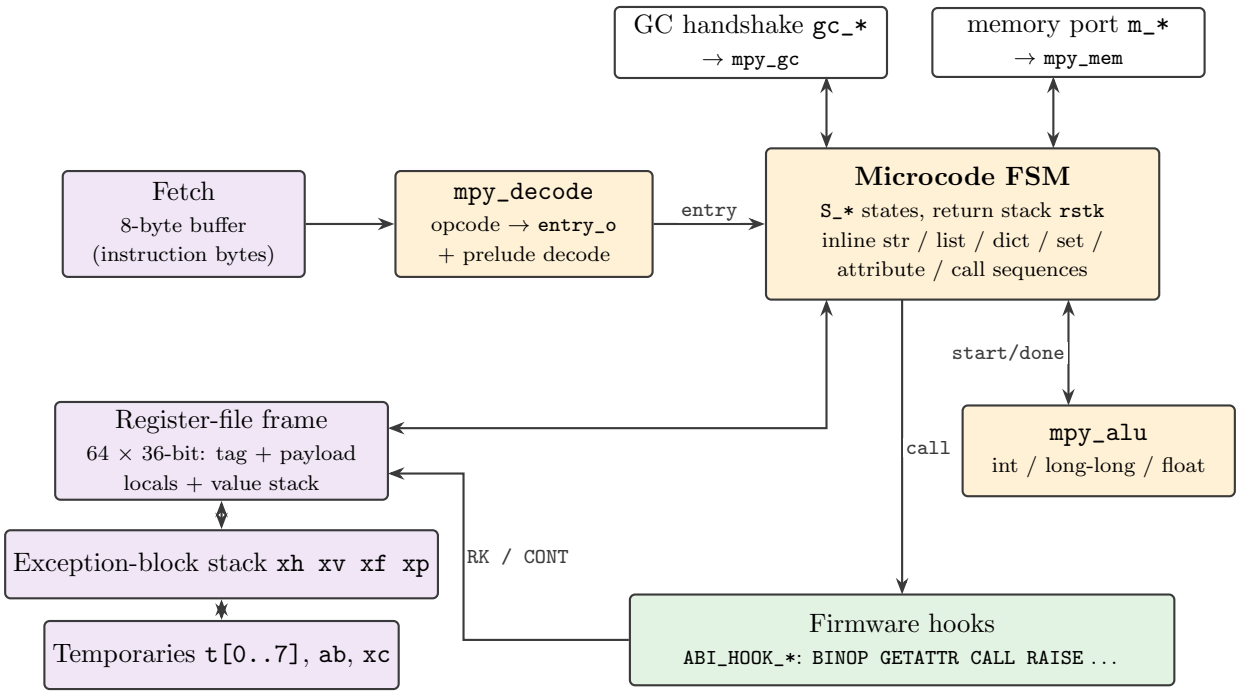


Figure 3: `mpy_core`. The microcode FSM sequences each opcode; instruction fetch and decode, the register frame, the ALU, and the two external handshakes are the only structural blocks. A hook returns through a return kind (`RK`) or continuation (`CONT`).

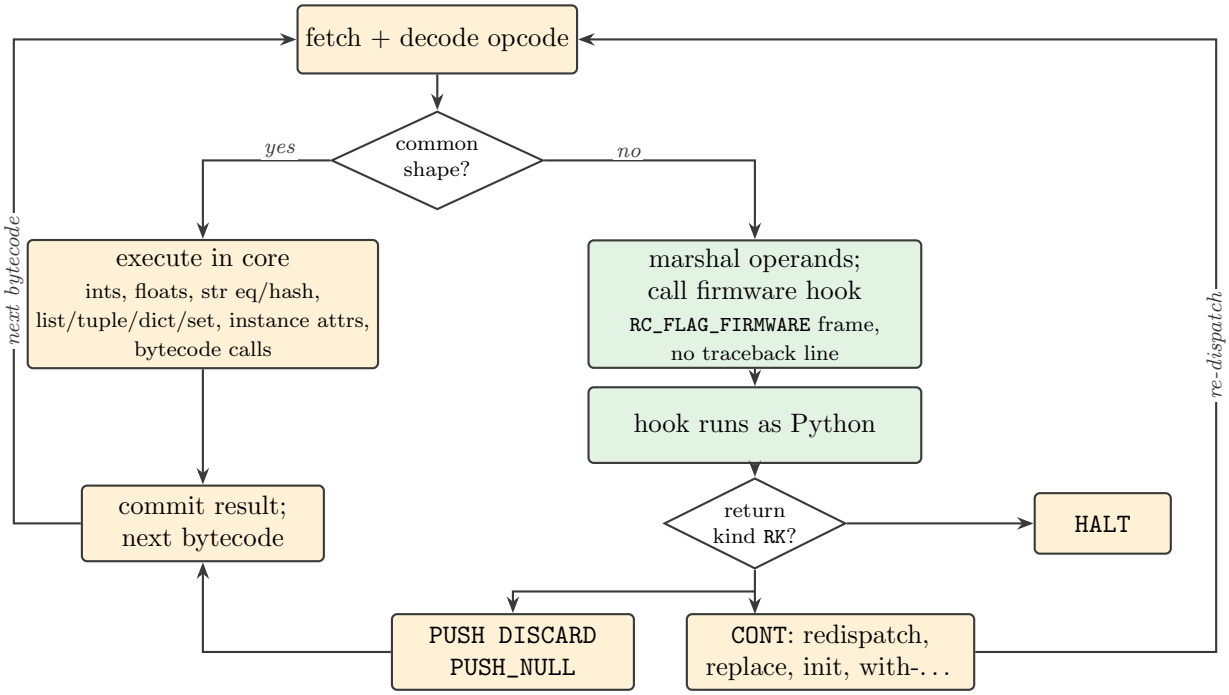


Figure 4: Opcode dispatch. A hook returns through a return kind `RK`; a continuation `CONT` (`REDISPATCH`, `REPLACE`, `INIT`, the `WITH_*` family) lets a hook finish by re-running the original opcode.

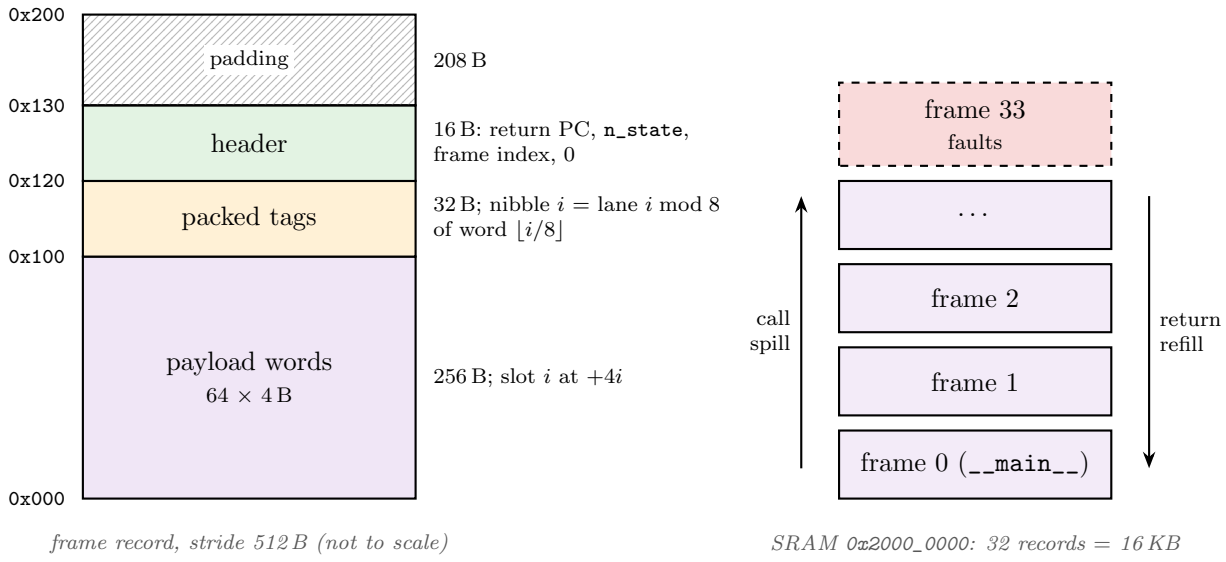


Figure 5: Frame record layout (left) and the 32-deep call stack in the SRAM frame reserve (right). A spill writes every physical register; tags for these slots live in the packed words of the record, not in a second copy.

4 Memory system and value encoding

There is no DRAM and no cache. SRAM is one cycle; flash adds `MPY_FLASH_WAIT_STATES` (default 1). Every 32-bit data word has a 4-bit tag held in a side array, eight tags to a word.

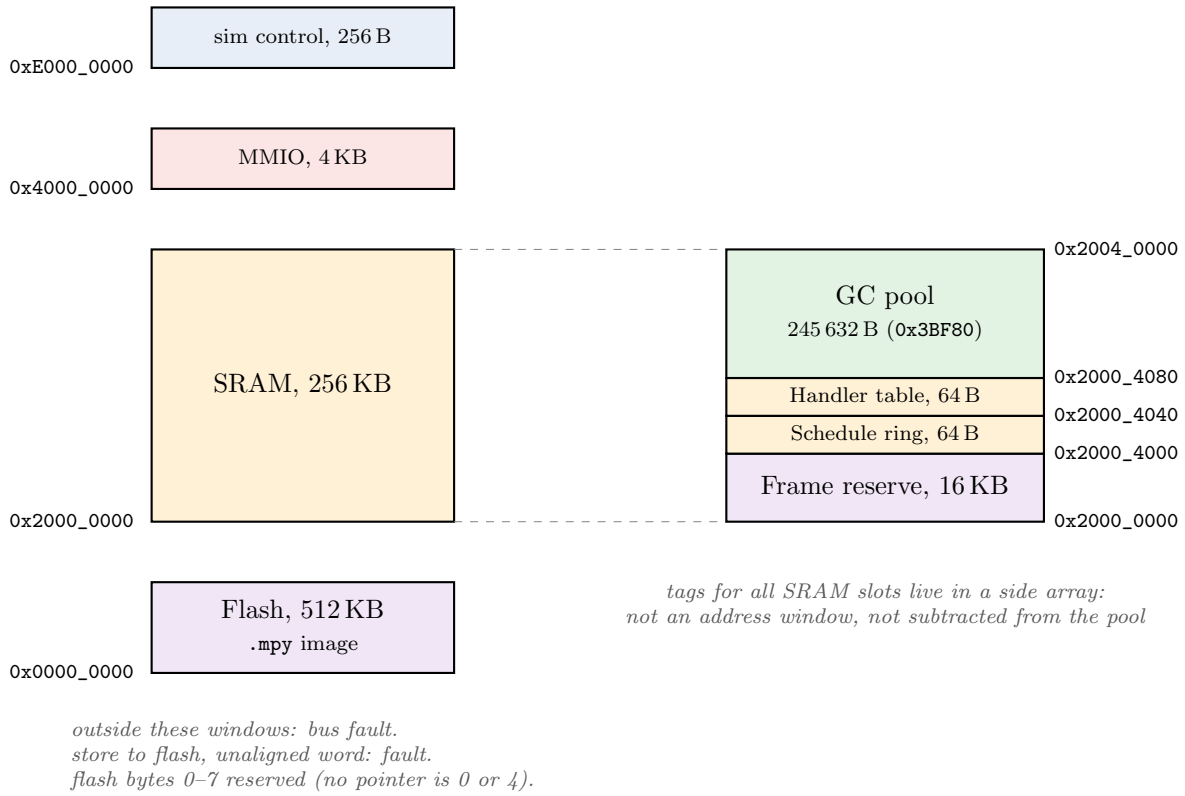


Figure 6: Address map. The frame-reserve tags are the packed words inside each frame record; the handler table is a GC root.

4.1 Tagged values

A value is a 4-bit tag beside a 32-bit payload; the register file is a single 36-bit array. Figure 7 groups the sixteen tag codes by what the payload means and whether the collector follows it.

4.2 Heap object layouts

5 Arithmetic datapath

`mpy_alu` is a small sequencer (`A_IDLE` → `A_MUL | A_DIV | A_POW | A_FPU` → `A_DONE`) around shared functional units. Single-cycle integer operations are answered at accept; multi-cycle units use a start/done/stall handshake. Floating point is evaluated in binary64 and narrowed to the binary32 that MicroPython values carry.

6 Garbage collector

The pool is divided into 16-byte blocks tracked by a two-bit allocation table (`FREE`, `HEAD`, `TAIL`, `MARK`). Allocation is next-fit with zeroing. A collection marks from two root ranges, scans objects from an explicit stack, and sweeps. On a mark-stack overflow, the stack drains, every marked block is rescanned, and the loop repeats until a pass is overflow-free.

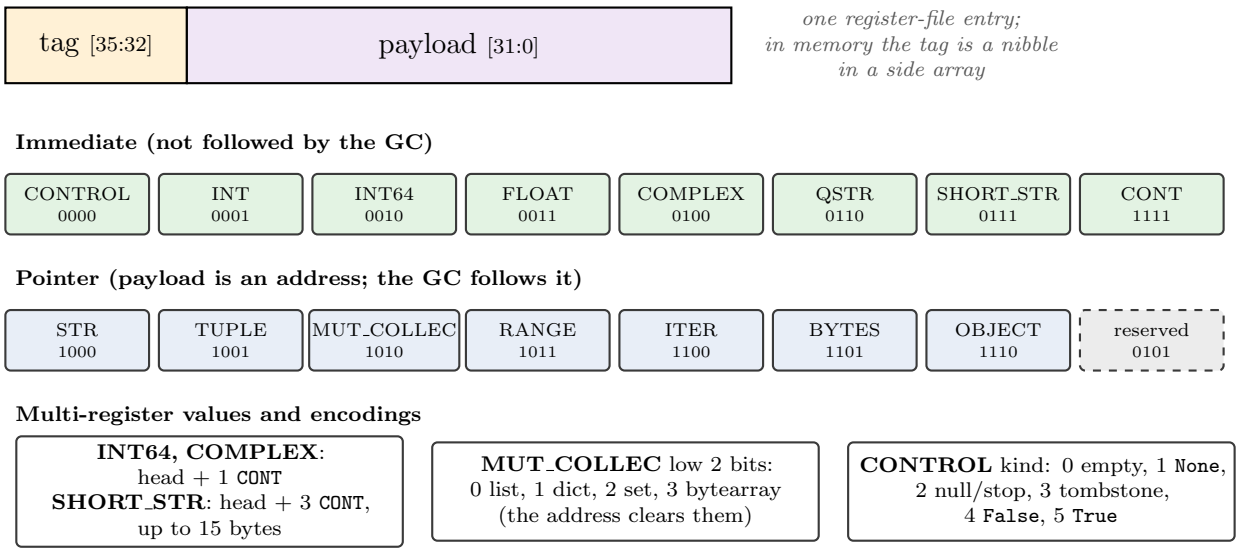


Figure 7: Value encoding. `False` and `True` are `CONTROL` kinds 4 and 5 and double as integers 0 and 1 in arithmetic and hashing.

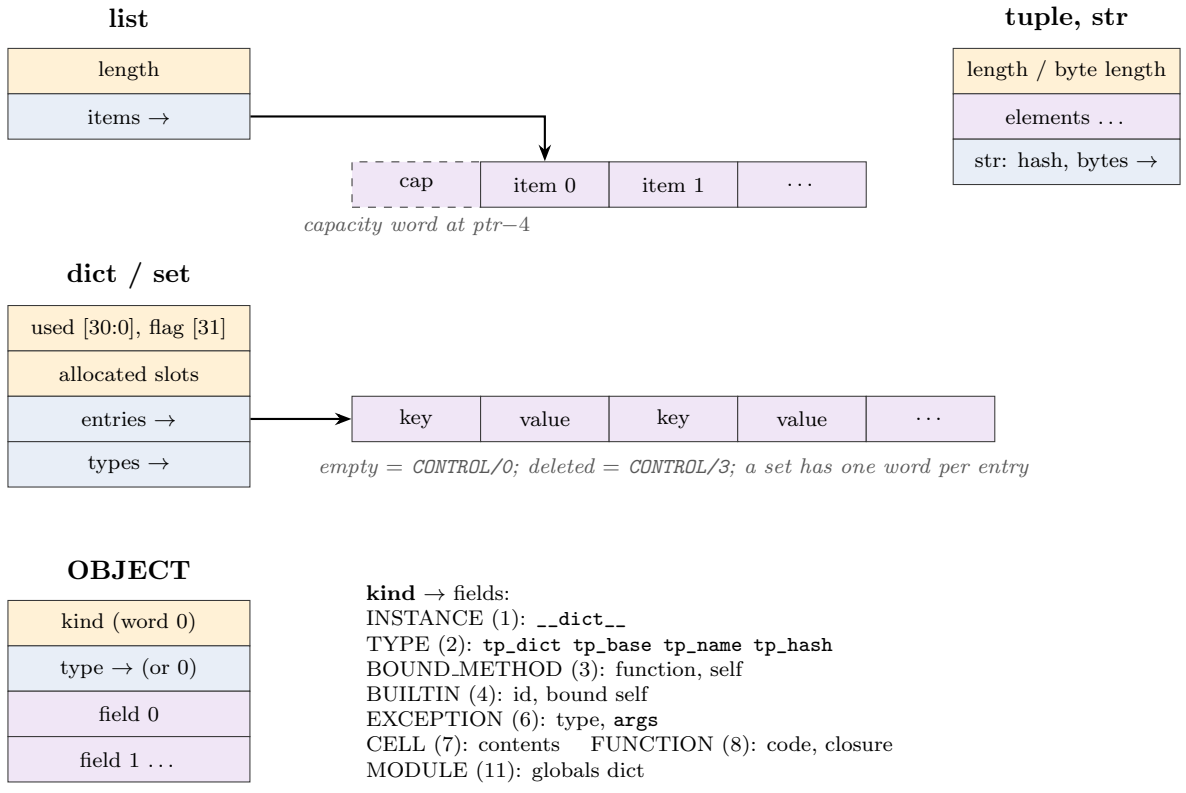


Figure 8: Heap object layouts. Blue cells are pointers the collector follows (gated by their tag nibble), orange cells are scalar headers it does not, purple cells are tagged value slots. Objects are 4-byte aligned and never move.

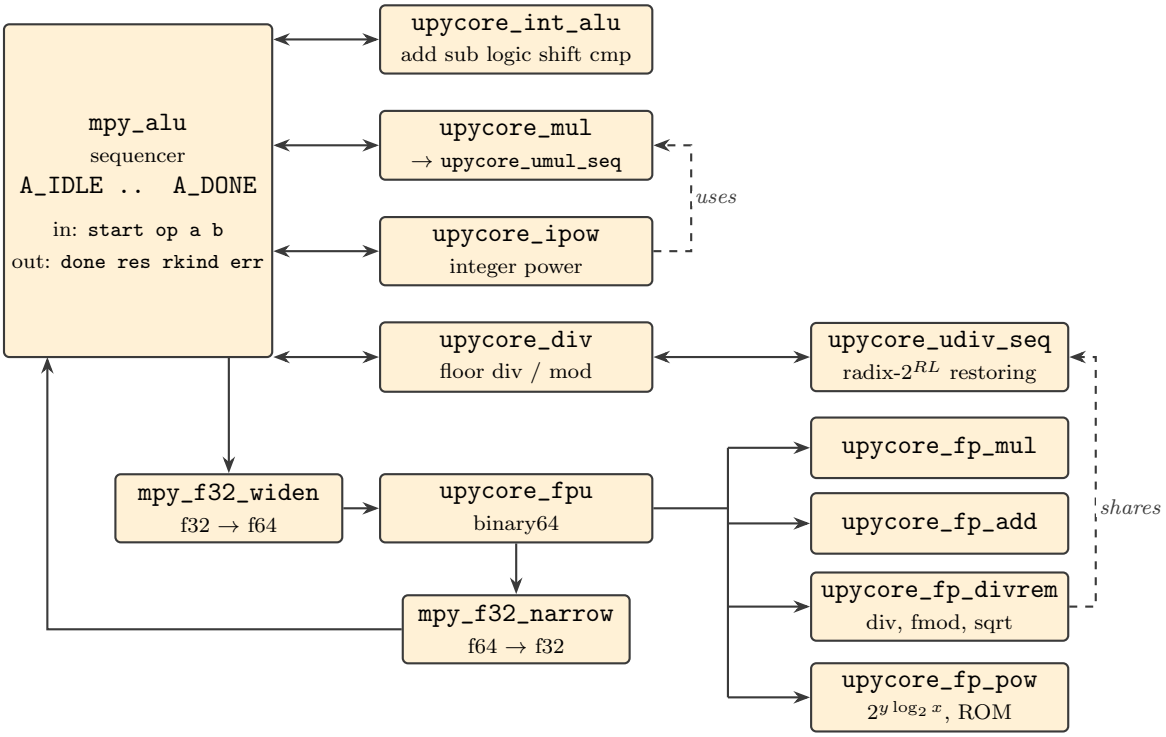


Figure 9: Arithmetic hierarchy. The integer unit is combinational; multiply, divide, and power are sequenced; the FPU composes add / mul / div-rem / pow pipelines into every float operation, with widen and narrow stages at the ALU boundary.

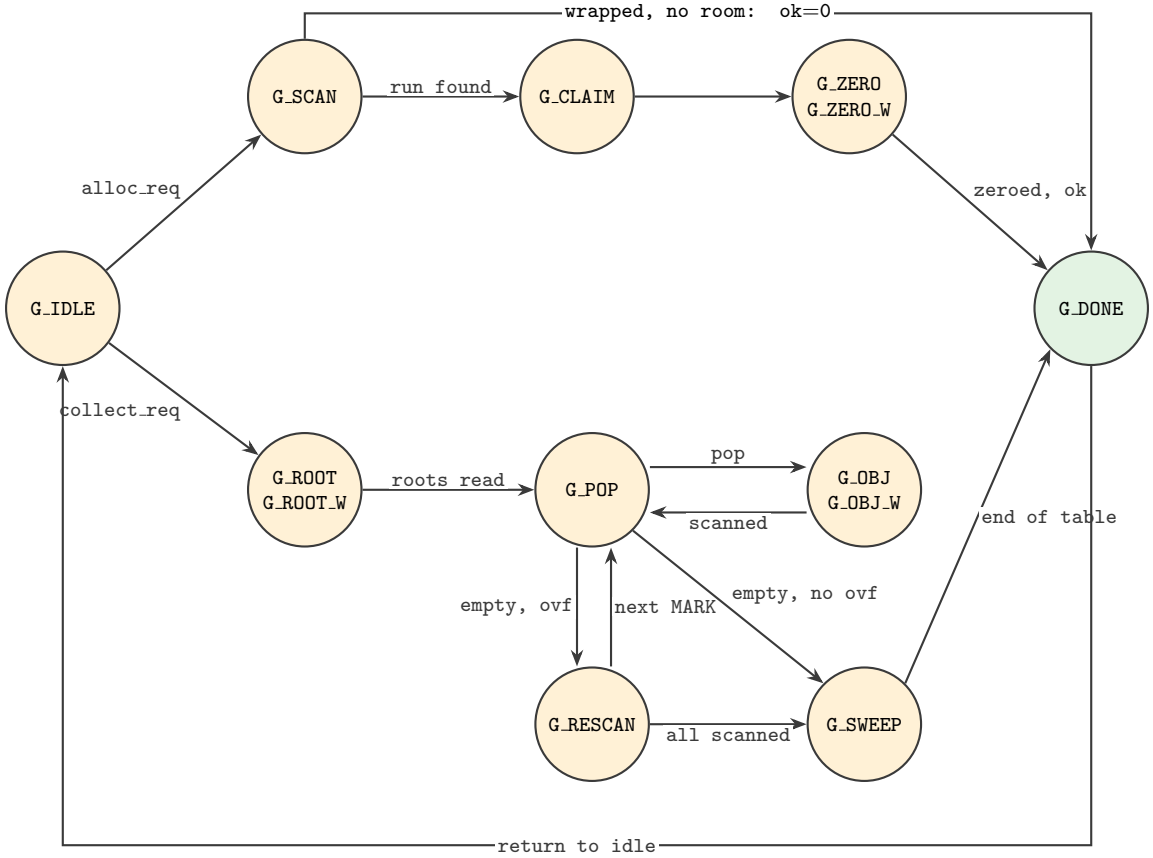


Figure 10: `mpy_gc` state machine. Top row: allocation. Lower rows: mark-sweep collection. State names follow the RTL `gstate_e` enum.

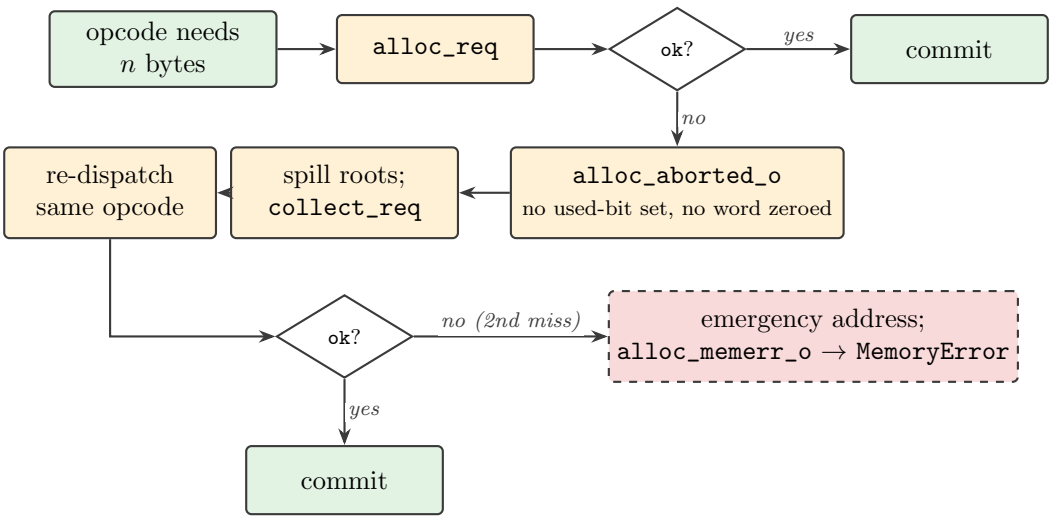


Figure 11: Allocation-failure protocol between `mpy_core` and `mpy_gc`. The opcode aborts *before* its first architectural commit, so the re-dispatch after collection is exact. The second `alloc_req` reuses the same size; its miss returns the pre-allocated emergency object.

7 Interrupts and peripherals

`mpy_periph` and `mpy_irq` implement the v1 MMIO register model (`mpy_peripherals.md`). They have a standalone testbench and an executable Python model (`mpy_periph_model.py`) but are *not yet instantiated* by `mpy_soc`; the hart will drive `bc_boundary` and `jump` pulses and map stores in the handler-table range to `handler_we`.

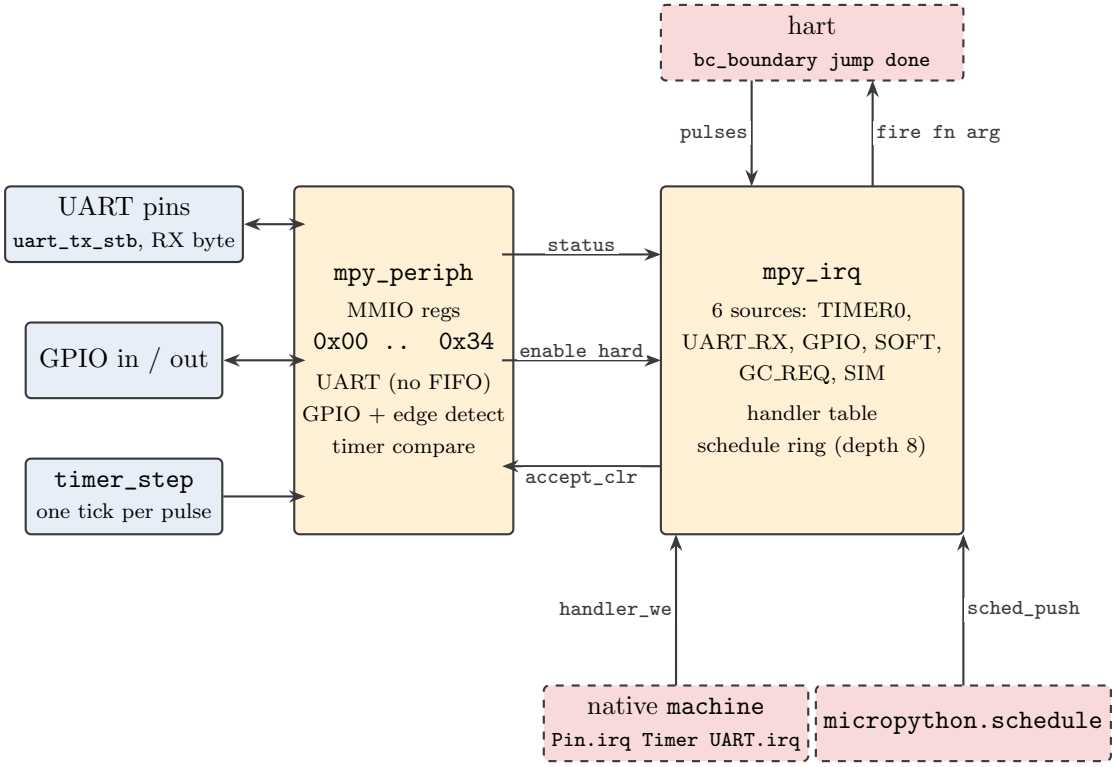


Figure 12: Peripheral and interrupt block. Red dashed blocks are the intended hart-side and software consumers; they are not connected in the current tree.

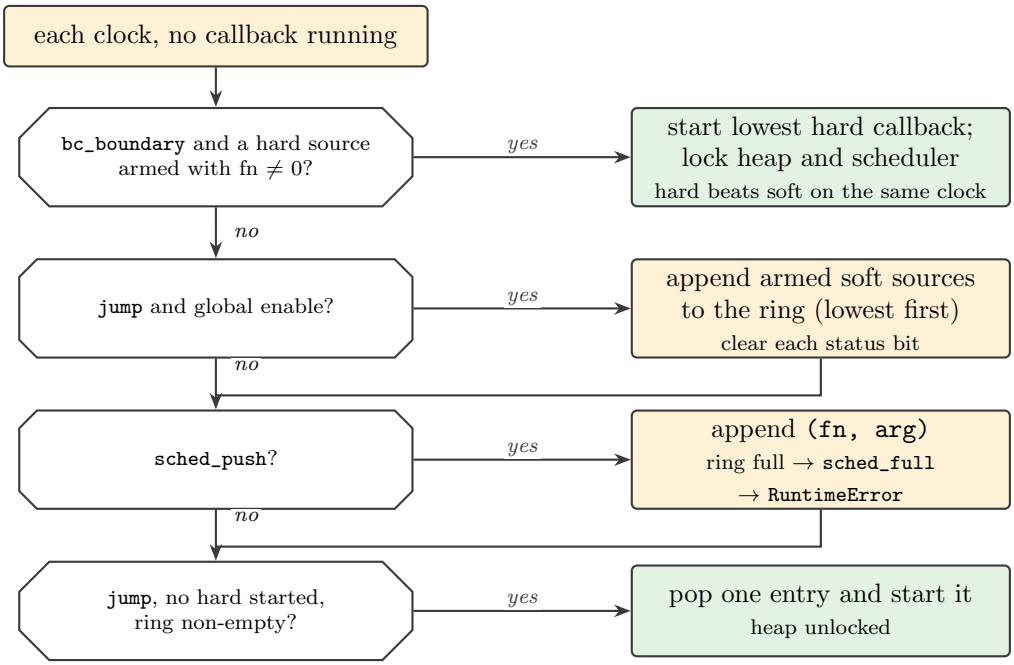


Figure 13: Per-clock callback selection in `mpy_irq`, in priority order. Soft device IRQs and `micropython.schedule` share one ring and run one callback at a jump opcode. A callback that was queued on an empty ring runs on that same jump. There is no nesting.

8 Verification map

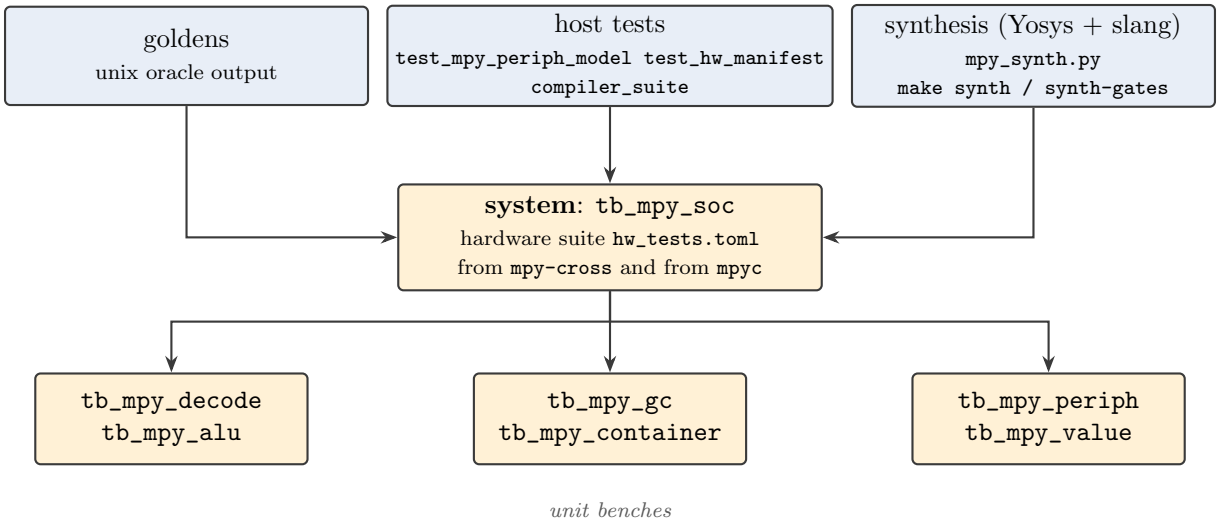


Figure 14: Test layering. System-level runs compare stdout, stderr, and exit status with the oracle's golden files, from both compilers; unit benches exercise the decoder, ALU, collector, container engine, and peripheral block in isolation; a Yosys flow elaborates every top and gate-maps the logic tops.